SOROTI UNIVERSITY

SCHOOL OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

YEAR 3, SEMESTER 1

COURSE UNIT: WIRELESS SENSOR NETWORKS

LECTURER: DR. STANLEY MUGISHA

Laboratory 1: Design, Implementation and Calibration of a Multi-Sensor Edge Node

Team members:

NAME	STUDENT NUMBER
Joseph Walusimbi	2401600068
Patience Clare Esaete	2401600415
Ritah Kobusingye	2401600224

Table of Contents

1. Objective and Competency 3
2. System Architecture and Hardware 3
3. Sensor Interfaces and Sampling Design 5
4. Sensor Characterization and Data Quality 5
5. Edge Processing and Local Decision Logic 9
6. Python Data Analysis and Visualization 12
7. Discussion and Engineering Interpretation 14
8. Limitations and Recommended Improvements 16
9. Advanced Challenge (optional) 17
10. Conclusion 17
11. References 17
12. Repository Link and File Structure 18

1. Objective and Competency

The objective of this lab was to build a low-cost sensor node using an ESP32 that collects data from three types of sensors (temperature/humidity, light and motion). The node checks that the readings make sense, calibrates the temperature, does some processing on the device itself (moving average, rate of change and threshold detection), makes a simple decision, and sends out timestamped CSV data that we then analysed in Python. The competency demonstrated is designing a complete and reliable sensing node, from connecting the sensors to analysing the data.

2. System Architecture and Hardware

The node has five main parts. (1) Sensing: a DHT11 temperature/humidity sensor on GPIO4, an LDR with a resistor on GPIO34, an HC-SR501 PIR motion sensor on GPIO27, and an LM35 temperature sensor on GPIO35, which we used as the reference for calibration. (2) Processing: the ESP32 firmware, which validates, calibrates and filters the readings and makes the decision. (3) Local indication: a status LED on GPIO2 and a buzzer on GPIO25. (4) Communication: a USB serial connection that sends one CSV row per second at 115200 baud. (5) Analysis: Python and Jupyter notebooks on a PC.

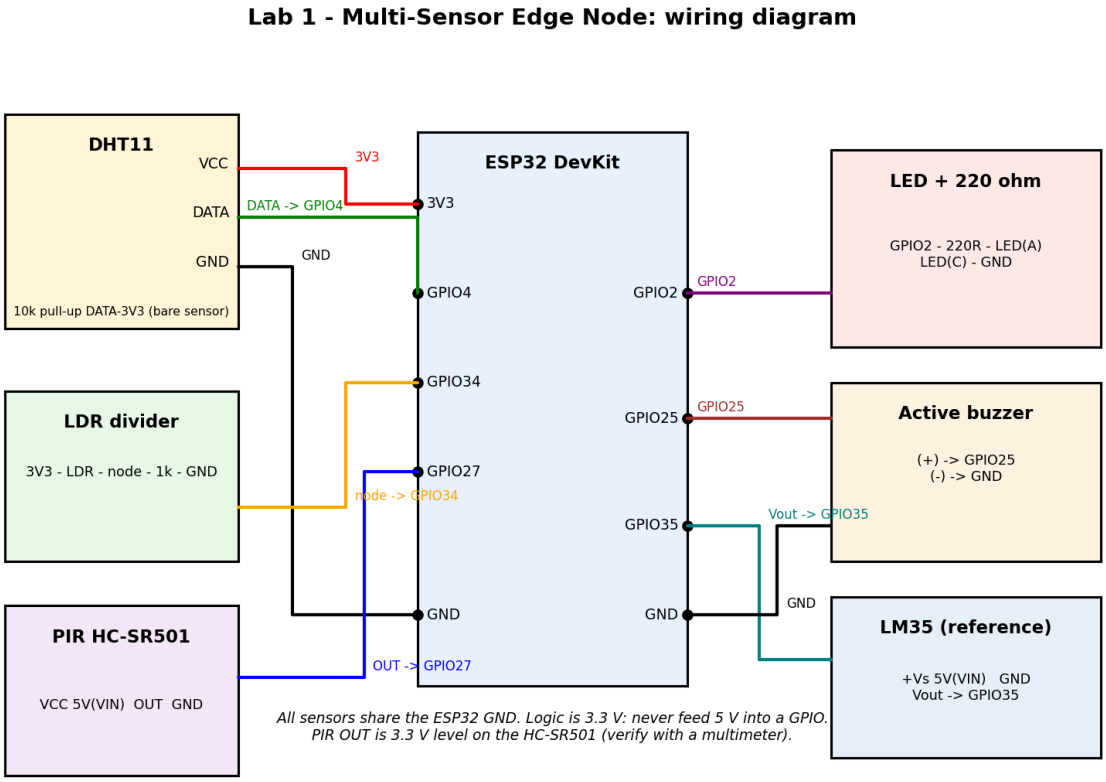


Figure 1(a): Wiring diagram of the node.



Figure 1(b): The ESP32 board used (DFRobot FireBeetle Board-ESP32 V4.0).

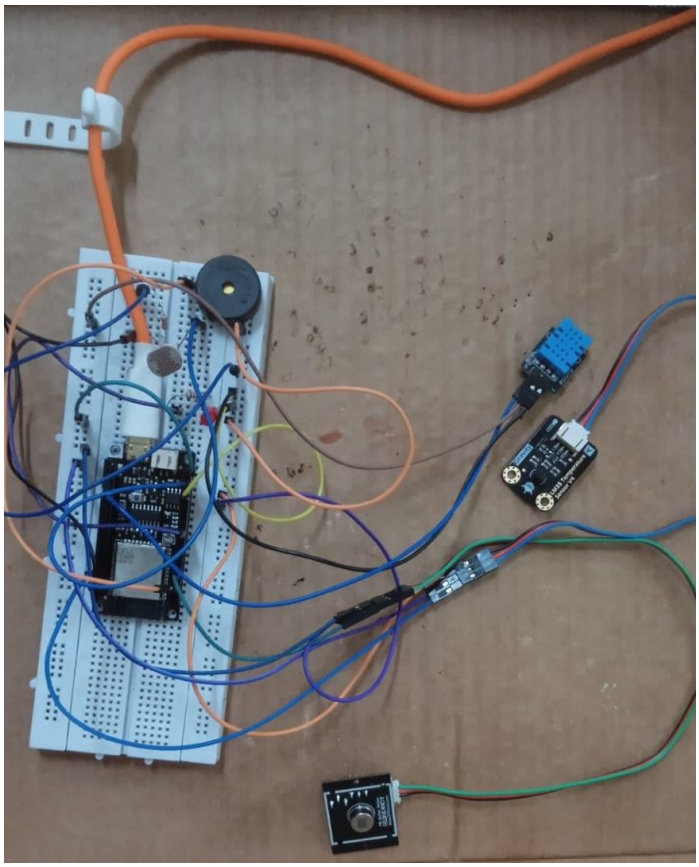


Figure 1(c): The assembled prototype. The ESP32 is on a breadboard with the LDR and buzzer, and the DHT11 and LM35 modules are connected with jumper wires.

3. Sensor Interfaces and Sampling Design

- DHT11 [2], read with the Adafruit DHT library [4]: it uses a single data wire and is slow, so we read it every 2 s. On the rows in between, the last value is kept and the dht_fresh column is 0.
- LDR: a voltage divider read by the ESP32's 12-bit ADC. We take 8 readings, average them and scale the result to a value between 0 and 1.
- PIR (HC-SR501 [3]): a digital output read every second. It is ignored in the status for the first 30 s after start-up while the sensor settles.
- Simulation (Wokwi, https://wokwi.com/projects/476511793297040385): the same firmware runs in the simulator with two changes. Wokwi only has a DHT22, so DHT_TYPE was set to DHT22 (with the DHT11 setting the readings were wrong), and calibration was switched off (gain 1.0, offset 0.0) because the fitted values belong to our physical DHT11. An analog NTC sensor stands in for the LM35, so ref_temp in the simulation is not a real temperature. The Wokwi online compile was sometimes delayed by a busy build queue.
- Main loop: the loop runs at 1 Hz using a timer based on millis() that does not drift and still works when the timer overflows.

CSV columns: timestamp, temp, humidity, light, motion, temp_cal, temp_filt, delta_temp, status, valid, dht_fresh, event and ref_temp. The first five match the example in the lab guide.

4. Sensor Characterization and Data Quality

We recorded one 10-minute log (17:15:37 to 17:25:36) with 600 rows at 1 Hz. All rows were valid (0 invalid DHT readings), with 19 events, 574 NORMAL rows and 26 MOTION rows. Table 1 shows the statistics from analysis.ipynb.

Quantity	n	Mean	Min	Max	Std
Raw temperature (°C)	300	30.390	30.300	30.500	0.057
Calibrated temperature (°C)	300	32.480	32.380	32.610	0.065
Filtered temperature (°C)	300	32.480	32.380	32.610	0.063
Humidity (%)	300	54.150	53.500	54.600	0.431
Light (normalised 0–1)	600	0.6275	0.1358	0.9274	0.2900

PIR (0/1)	600	0.043	0	1	0.204
LM35 reference (°C)	600	32.220	31.860	32.780	0.110

Table 1: Descriptive statistics of the 10-minute log. The DHT rows count only its 300 fresh readings.

Baseline: in the first 5 minutes (first 300 rows) the calibrated temperature was 32.52 °C (std 0.05, range 32.49 to 32.61 °C), the humidity was 54.52 % (std 0.05, range 54.40 to 54.60 %) and the LM35 read 32.24 °C (std 0.12). The baseline was not completely undisturbed. The PIR triggered on 13 rows in this period (10 events, for example at 17:19:52 to 17:20:03) because of movement in the room.

LDR: the updated 600-sample dataset shows that the light channel was no longer stuck at full scale. The normalised readings ranged from 0.1358 to 0.9274, with a mean of 0.6275 and a standard deviation of 0.2900. The graph shows several distinct operating regions: high readings near 0.91 at the beginning, a short decrease around samples 25–38, a relatively steady region around 0.61–0.64 through approximately sample 200, and another high region around 0.91 from about samples 205–405. After sample 410, the readings fall sharply and remain mostly around 0.14–0.22, with several short spikes, before rising to about 0.63 near the final samples. These changes demonstrate that the LDR channel responds with clear variation across the recorded sequence. However, the values are normalised ADC readings, not calibrated illuminance in lux. Since the illumination was not measured independently during acquisition, the graph supports interpretation of relative changes only; it does not establish the exact light intensity or prove the cause of each transition.

To show how the light channel should behave, we used the Wokwi simulation (an ideal simulated photoresistor, not measured hardware data). Changing the illumination slider changes the light value from the firmware: 0.9875 at 0.2 lux (dark), 0.3509 at 240 lux (room light) and 0.0095 at 75858 lux (bright). In this module the value goes down as the light goes up (Figure 2).

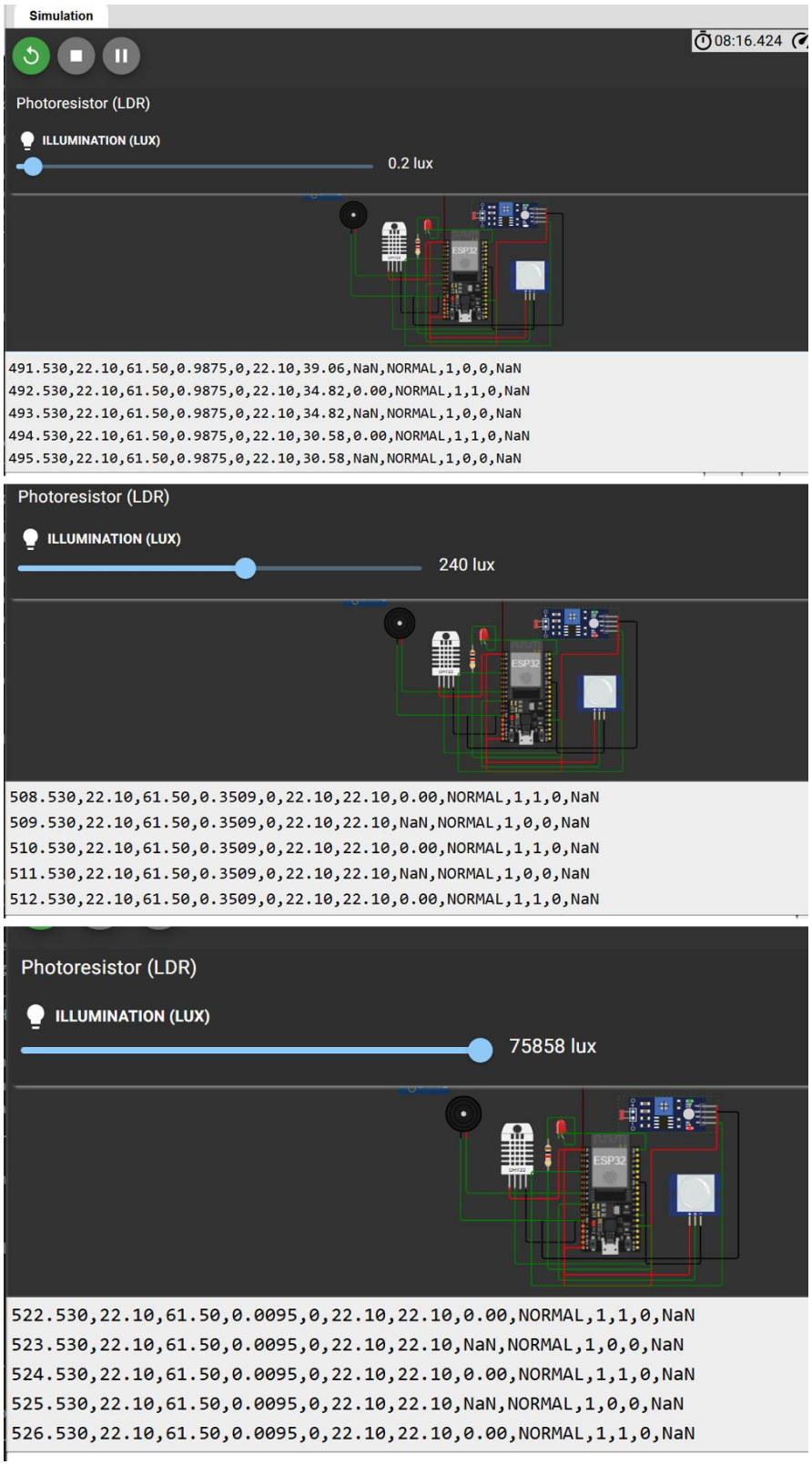


Figure 2: Wokwi simulation of the light channel at 0.2 lux (top, light = 0.9875), 240 lux (middle, 0.3509) and 75858 lux (bottom, 0.0095).

PIR: the PIR gave 26 motion rows in 18 separate bursts, mostly 1 to 2 s long, spread over the run (for example at 17:15:46, 17:16:47 and 17:18:24, a group at 17:19:52 to 17:20:03, and 17:25:24 to 17:25:30). Each burst was recorded as a MOTION status with the event flag set on its first row. We did not log the exact times of our walk-ins, so we cannot match each trigger to a walk-in or say how repeatable the sensor was or whether it gave false triggers.

DHT: over the 10 minutes the raw temperature stayed between 30.3 and 30.5 °C and the humidity between 53.5 and 54.6 %, so the conditions were stable. There were no invalid readings (0 out of 300). The sensor is slow (a new value every 2 s) and coarse (steps of 0.1 °C). The raw DHT reading was about 2 °C lower than the LM35, which the calibration below corrects.

The 10-minute log had no invalid readings, so we tested the invalid-data handling in the Wokwi simulation by disconnecting the DHT22 data wire. Every row is still printed, with NaN in the DHT columns, status INVALID and valid = 0, while the light and PIR columns keep updating (Figure 3). Invalid rows are flagged and never dropped.

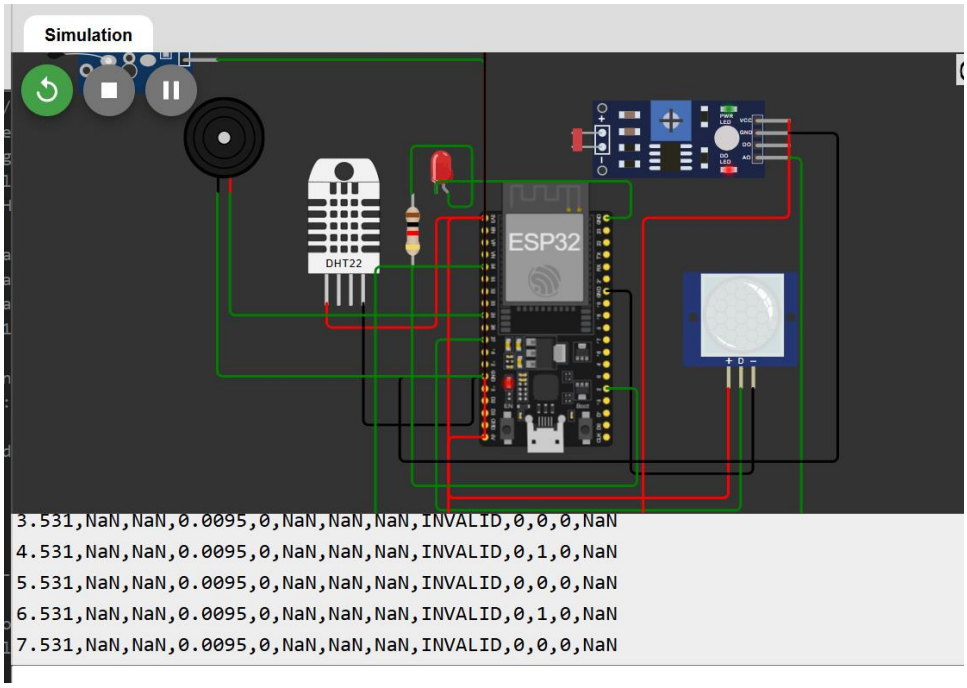


Figure 3: Wokwi simulation with the DHT22 data wire disconnected. The rows are kept as INVALID with valid = 0 and NaN values.

Calibration: we did not have a calibrated thermometer, so we used an LM35 temperature sensor (10 mV per °C) as the reference. It is read by the ESP32 ADC and logged in the ref_temp column. We placed both sensors next to each other at several temperatures, and calibrate.py averaged each point over 60 s and fitted temp_cal = gain × temp + offset using

least squares. The LM35 is only accurate to about ±0.5 °C and the ESP32 ADC adds some error at low voltages, so this is a comparison with a second sensor and not a proper reference. The calibrated accuracy is therefore about ±1 °C at best.

Point	LM35 reference (°C)	DHT raw (°C)	Error = raw - ref (°C)	DHT std (°C)	n
1	33.43	31.61	-1.82	0.03	31
2	35.20	33.15	-2.05	0.20	30
3	35.85	33.03	-2.82	0.16	30
4	37.88	34.52	-3.36	0.48	30
5	39.25	36.46	-2.79	0.19	30
6	39.50	37.05	-2.45	0.09	30

Table 2: Calibration points (each is a 60 s average with the sensors side by side).

Fitted model (6 points, $R^2 = 0.957$): temp_cal = 1.11587 × temp_raw − 1.42758. Before calibration the DHT read 2.55 °C too low on average (MAE 2.55 °C, RMSE 2.60 °C). After calibration the MAE is 0.40 °C and the RMSE is 0.46 °C on the same points. With leave-one-out validation the MAE is 0.60 °C and the RMSE is 0.68 °C. The fit is shown in Figure 13.

Excluded point: we recorded a seventh point (LM35 36.50 °C, DHT 35.67 °C) but left it out of the fit because the temperature was still changing during its 60 s window (DHT std 1.01 °C, much higher than the other points). The two sensors were therefore not at the same steady temperature. If it had been included, the fit would have given a gain of 1.024 and an offset of +1.47 °C with $R^2 = 0.86$. We mention it here so that it is not hidden.

Valid range: the calibration points cover only about 31.6 to 37.0 °C on the raw DHT (33.4 to 39.5 °C on the LM35). The calibration is only valid in this range. Outside it the calibrated value is an extrapolation of the line and the error is unknown, so it should not be trusted, for example at normal room temperature (about 20 to 25 °C) or below.

5. Edge Processing and Local Decision Logic

- Validation: each reading is checked to be a number, to be inside the allowed range (temperature −40 to 80 °C, humidity 0 to 100 %) and not to jump by more than 5 °C between reads. Invalid readings are never dropped. The row is still printed with NaN, status INVALID and valid = 0, because the lab guide says to record unexpected behaviour.
- Calibration: temp_cal = gain × temp + offset.
- Moving average: the calibrated temperature is averaged over the last 5 samples (temp_filt).

- Rate of change: delta_temp = temp_cal(t) − temp_cal(t−1), calculated between fresh DHT readings.
- Decision: the status is HIGH_TEMP when temp_cal goes above the threshold. A 0.5 °C hysteresis stops the status from switching on and off repeatedly near the threshold. MOTION comes from the PIR, MOTION_AND_HIGH_TEMP means both, and INVALID means the DHT reading failed validation. An event is the row where the status changes from NORMAL to something else.
- Indicators: the LED is on whenever motion or high temperature is detected. The buzzer beeps 250 ms on and 250 ms off while HIGH_TEMP is active, and gives one 150 ms chirp on a motion-only event. It does not block the code, so the 1 Hz sampling is not affected.

Threshold: the firmware uses T_HIGH = 35.0 °C (cleared at 34.5 °C). We set it above the room temperature so that the node does not alarm all the time. The lab guide example uses 30.0 °C. During bench testing with a temporary threshold of 30 °C, the HIGH_TEMP status, the LED and the pulsing buzzer worked. The 10-minute log has no HIGH_TEMP rows because the calibrated temperature (about 32.5 °C) stayed below 35 °C, so we demonstrated the threshold logic in the Wokwi simulation instead (Figures 4 to 7).

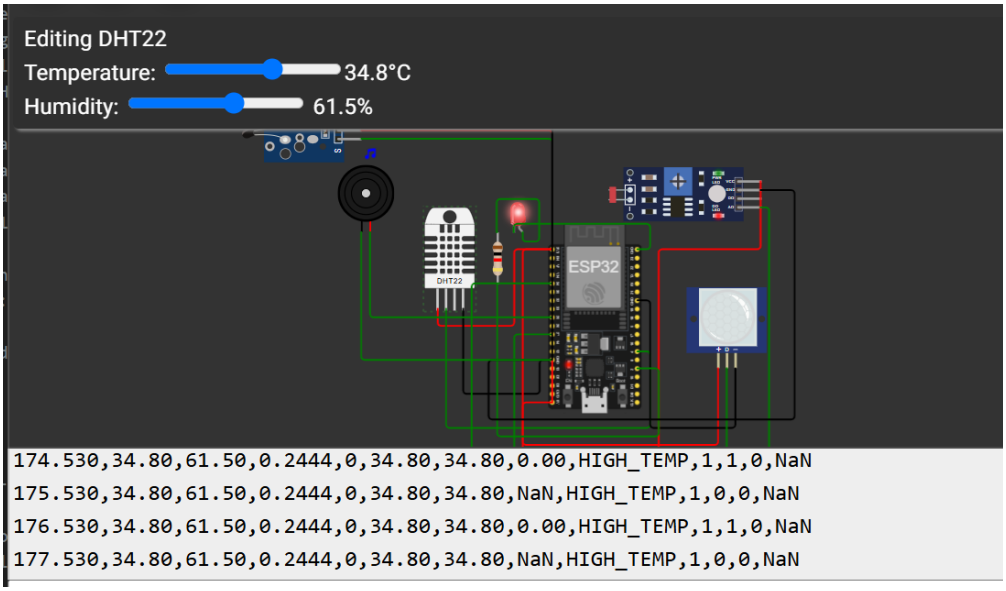


Figure 4: Wokwi simulation with the DHT22 set to 34.8 °C. The status is still HIGH_TEMP (LED on, buzzer active) because the hysteresis only clears it below 34.5 °C.

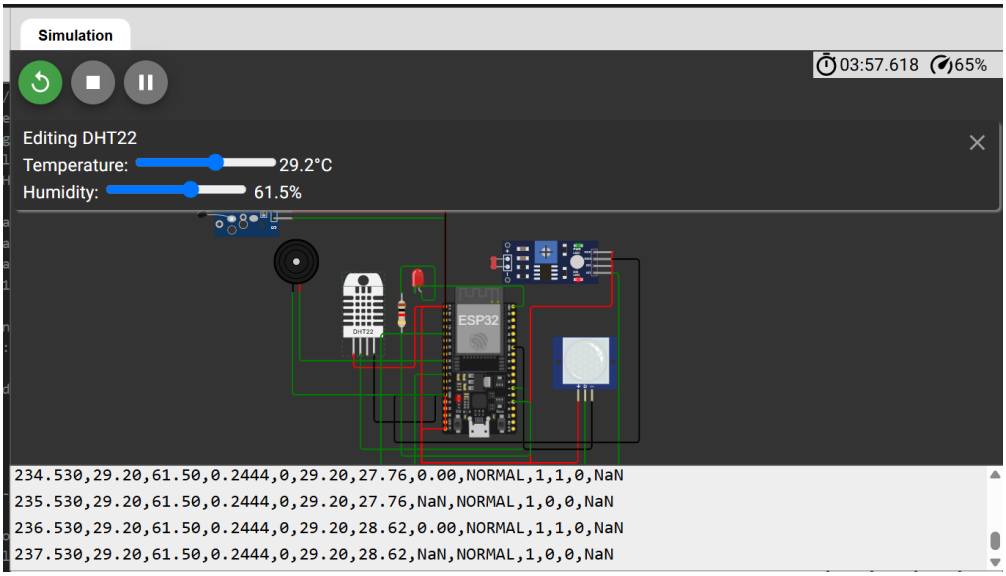


Figure 5: Wokwi simulation after lowering the temperature to 29.2 °C. The status returns to NORMAL, and temp_filt (27.76, then 28.62) is still catching up with temp_cal (29.20), which shows the lag of the 5-sample moving average.

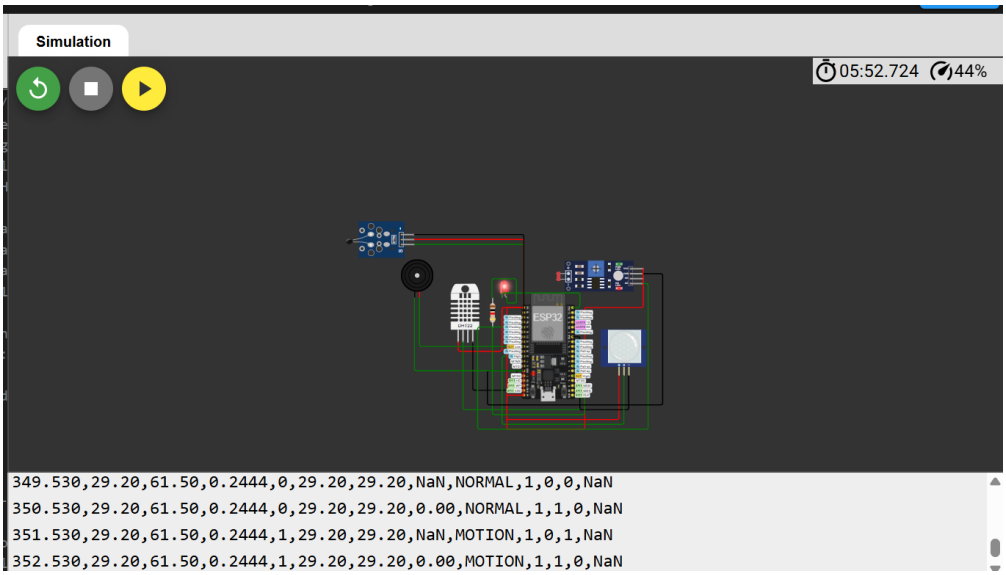


Figure 6: Wokwi simulation with the PIR triggered after the 30 s warm-up. The row changes to MOTION and event = 1 on its first row (351.530 s).

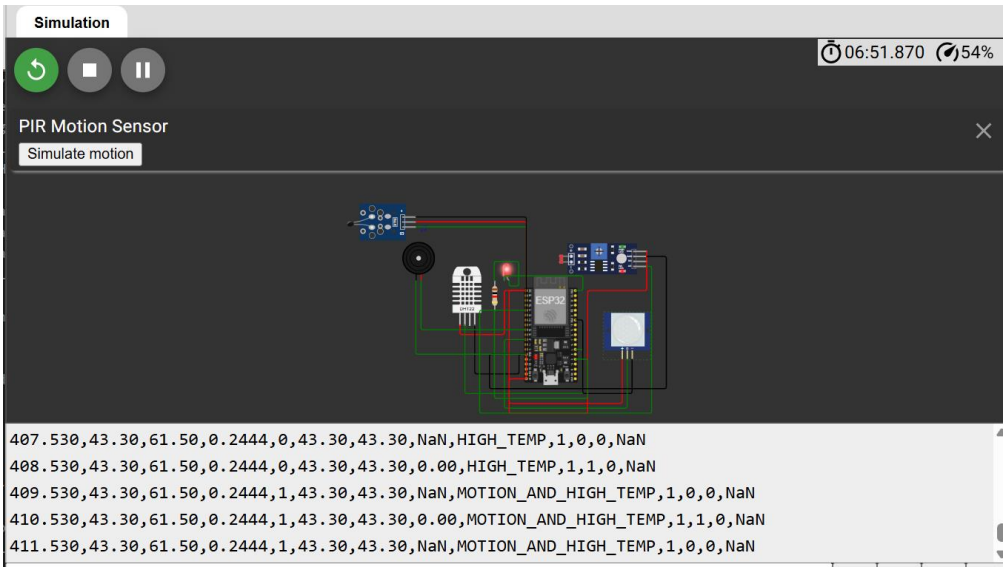


Figure 7: Wokwi simulation with the DHT22 at 43.3 °C and the PIR triggered. The status is MOTION_AND_HIGH_TEMP.

6. Python Data Analysis and Visualization

acquisition.ipynb saves at least 600 rows to data/raw/. analysis.ipynb then calculates the number and duration of observations, the mean, minimum, maximum and standard deviation, the number of invalid measurements and the number of events, and makes the plots below.

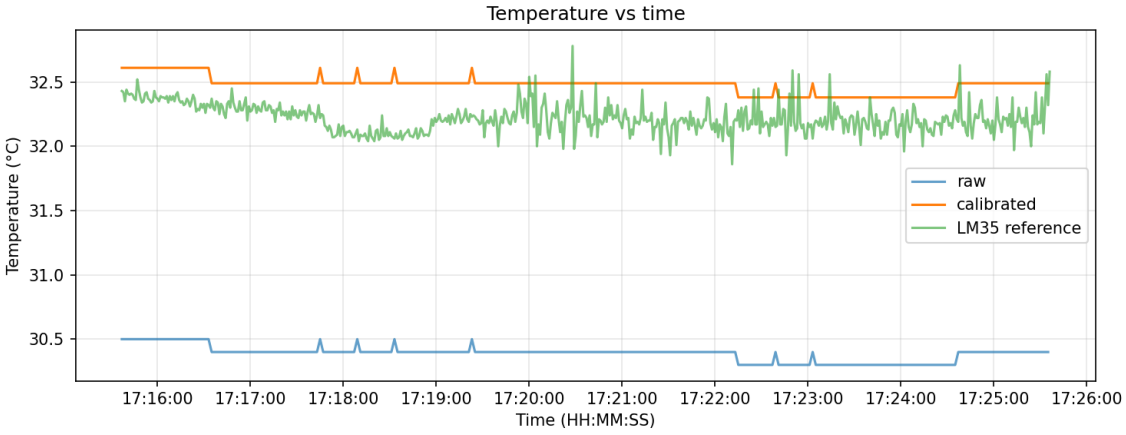


Figure 8: Temperature vs time (raw, calibrated and LM35 reference).

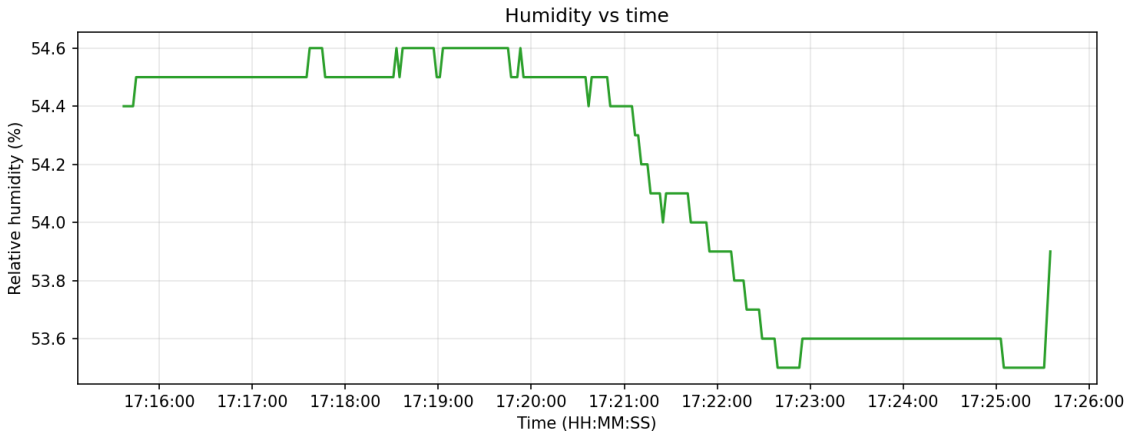


Figure 9: Humidity vs time.

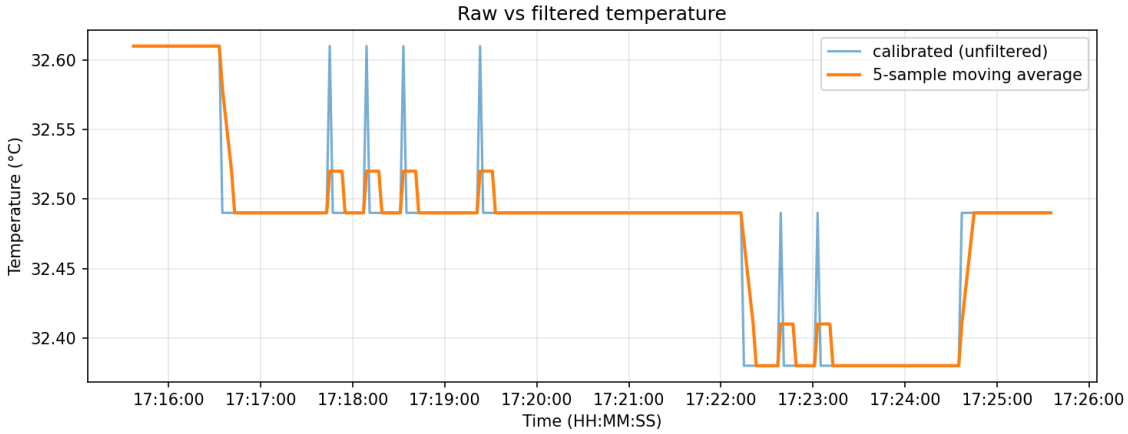


Figure 10: Calibrated temperature and its 5-sample moving average.

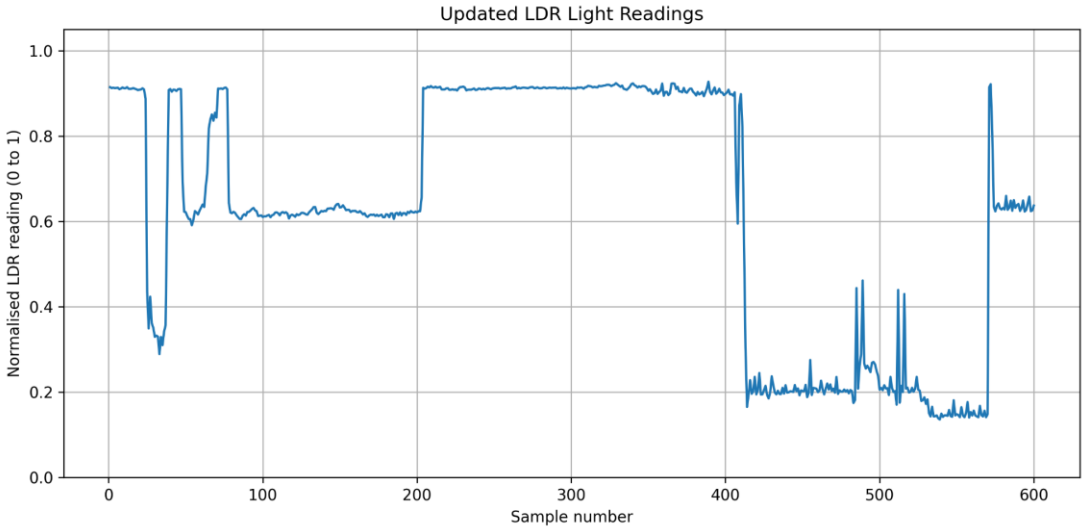


Figure 11: Updated LDR light readings across 600 samples (normalised values, 0–1).

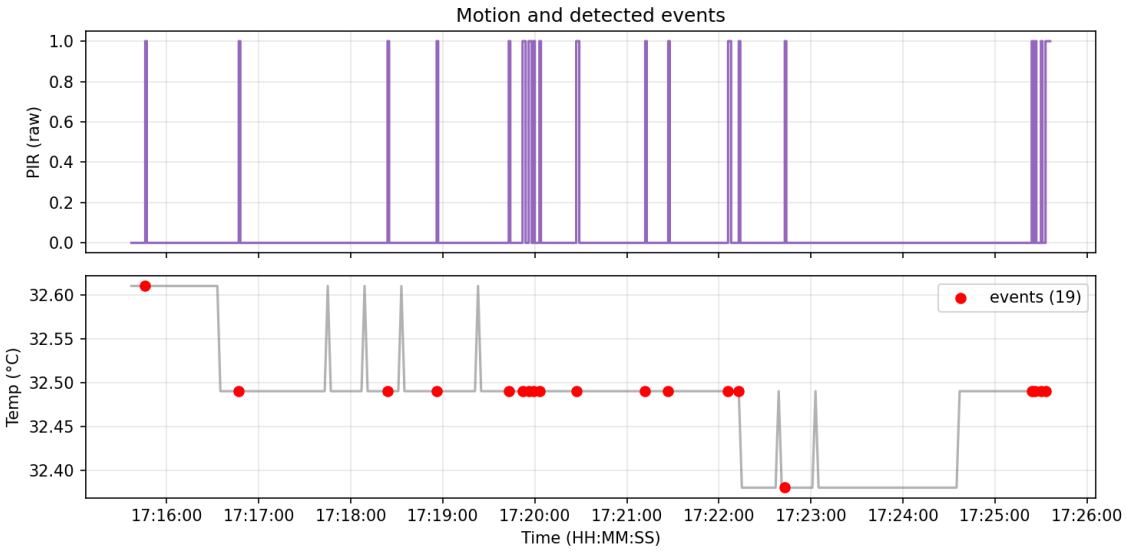


Figure 12: PIR motion and detected events.

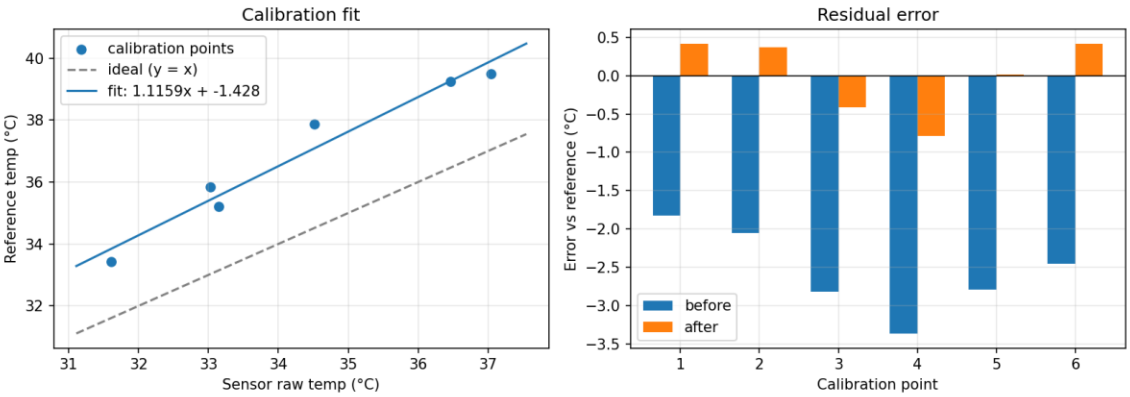


Figure 13: Calibration fit (DHT raw vs LM35) and the residual errors.

7. Discussion and Engineering Interpretation

1. Major subsystems and roles

The node has three sensing subsystems (DHT11, LDR, PIR) that turn physical quantities into signals. The ESP32 reads, validates, calibrates and filters them and makes the decision. The LED and buzzer give local feedback, the serial link provides the CSV data, and Python is used for analysis and plots.

2. Effective sampling rate of each sensor

The LDR and PIR are read at 1 Hz (once per loop). The DHT11 is read at 0.5 Hz (every 2 s). Over the 10-minute log we measured 1.002 Hz for the main loop and 0.501 Hz for fresh, valid DHT readings.

3. Which sensor showed the greatest variability and why

The LDR showed a wide spread in its normalised readings (standard deviation 0.2900, range 0.1358–0.9274), and its graph contains several clear transitions and short spikes. This is the largest numerical standard deviation among the reported channels, although standard deviations in different units should not be compared directly. Humidity varied by 0.43 percentage points around 54 %, while temperature varied by about 0.06–0.07 °C. The DHT is also coarse, with 0.1 °C steps and an update every 2 s. The PIR only gives 0 or 1 (26 active rows in 18 bursts), so its standard deviation says little. The LDR variation is consistent with changes in the light signal during acquisition, but without a separate lux meter or controlled illumination test, the exact illumination levels and causes of individual transitions cannot be confirmed.

4. Systematic error revealed by calibration

The DHT read lower than the LM35 reference by 2.55 °C on average over the 31.6 to 37.0 °C range. The fitted model is temp_cal = 1.11587 × temp_raw − 1.42758. The gain of about 1.12 shows a scale error (the gap gets bigger as the temperature rises), and the offset of −1.43 °C is a constant shift. This is only valid inside the calibrated range (about 31.6 to 37.0 °C raw). Outside it, the result is an extrapolation.

5. Did calibration reduce MAE and RMSE, and by how much

Yes. The MAE dropped from 2.55 °C to 0.40 °C (84 % lower) and the RMSE from 2.60 °C to 0.46 °C (82 % lower) on the calibration points. With leave-one-out validation, which is a fairer estimate for new points, the MAE is 0.60 °C and the RMSE is 0.68 °C. These errors are measured against the LM35, which is itself only accurate to about ±0.5 °C.

6. Effect of filtering on noise and responsiveness

The 5-sample moving average reduces the noise between samples. The standard deviation of the sample-to-sample change dropped by 69.5 % in our log, although the DHT only moves in steps of 0.1 °C. The cost is a delay of about (N − 1)/2 = 2 samples, which is about 4 s at the 2 s DHT update rate, so fast changes show up later.

7. What to process locally and what to transmit

Process locally: validation, calibration, filtering and threshold decisions. These are cheap to compute and let the node act even with no network. Transmit: short summaries, status changes and events, and periodic filtered values. Raw data should only be sent when diagnosing a fault or when it is needed to train a model.

8. Energy if every raw measurement were transmitted

The radio uses most of the energy. According to the ESP32 datasheet [1, Table 5-4], the chip draws about 240 mA when transmitting 802.11b (+19.5 dBm), 190 mA for 802.11g (+16 dBm) and 180 mA for 802.11n (+14 dBm), and 95 to 100 mA when receiving (3.3 V supply, transmit figures at 50 % duty cycle). Sensing and processing need much less. If every raw reading were transmitted, the radio would be active much more of the time, so the average current would be higher and the battery would drain faster. Sending only events and periodic summaries lets the radio sleep in between.

8. Limitations and Recommended Improvements

- The reference is an LM35 read by the ESP32 ADC, which is not a traceable thermometer (about ±1 °C overall). A calibrated thermometer, or ice-point and boiling-point checks, would give a stronger reference.
- The calibration covers only about 31.6 to 37.0 °C (6 points, with one excluded), so it is not validated at normal room temperature or outside this range. It should be extended with points over the whole operating range.
- We used only three sensors. The DHT11 is slow (one reading every 2 s) and coarse (datasheet accuracy of about ±2 °C and ±5 %RH [2]), so the calibration mainly corrects its bias.
- The LDR readings now show a clear response range (0.1358–0.9274), rather than remaining fixed at full scale. However, the readings are normalised ADC values and have not been calibrated against a lux meter, so they must not be reported as illuminance. A useful next improvement is to record readings under controlled dark, room-light and bright conditions while measuring illuminance with a reference meter. The voltage-divider wiring and resistor value should also be checked and documented so that the relationship between ADC value and light level is repeatable.
- The PIR needs about 30 s to warm up and its range depends on the environment.
- Data only leaves the node through the wired serial link, with no buffering. A local buffer and MQTT could be added for wireless operation.
- The single chirp on a motion-only event is implemented but we did not confirm it by listening or with a simulation screenshot. The MOTION status and event flag were confirmed (Figure 6).

- We recorded only one 10-minute log, at one room temperature (about 32 °C). Although the updated LDR trace varies clearly, we did not record simultaneous reference illuminance values or mark the times when the lighting conditions were deliberately changed. The exact light levels and repeatability of the LDR response therefore remain unverified. We also did not log the times of the walk-ins.

9. Advanced Challenge (optional)

Data volume estimate for changing the sampling interval from 1 s to 5 s: one CSV row is about 62 bytes. At 1 Hz that gives 86,400 rows per day (about 5.4 MB per day). At 0.2 Hz it gives 17,280 rows per day (about 1.1 MB per day), which is five times less data. The radio and CPU would also be active for about a fifth of the time, so the communication energy would drop by up to about five times. It would not drop by exactly five because the node still uses some current while it sleeps.

We did not implement the IMU, the local data buffer or MQTT parts of the advanced challenge.

10. Conclusion

We built an ESP32 edge node with three sensor types (DHT11 temperature/humidity, LDR and PIR) and an LM35 reference. The node validates and calibrates the readings, filters the temperature with a 5-sample moving average, calculates the rate of change, uses a hysteresis threshold and event detection, and drives an LED and a buzzer. We logged 600 valid rows at 1 Hz as a timestamped CSV and analysed them in Python. Calibration against the LM35 reduced the DHT temperature error from 2.55 °C to 0.40 °C (MAE on the calibration points, 0.60 °C with leave-one-out), but it is only valid for about 31.6 to 37.0 °C. The updated LDR data varied from 0.1358 to 0.9274 on the normalised 0–1 scale, with clear changes across the 600 samples; this shows that the light channel was not stuck at full scale in the updated dataset. The readings are not calibrated in lux, so their interpretation is limited to relative changes. The PIR gave 18 motion bursts (19 events overall) and the log had no invalid readings. The main lessons were to keep invalid data visible, that the reference sensor limits the accuracy that can be achieved, and that sensor outputs need appropriate calibration before being interpreted as physical units.

11. References

[1] Espressif Systems, "ESP32 Series Datasheet," Version 5.3, 2026. [Online]. Available: https://documentation.espressif.com/esp32_datasheet_en.pdf . Accessed: Sep. 25, 2026.

[2] Aosong (Guangzhou) Electronics Co., Ltd., "Temperature and humidity module DHT11 product manual," V1.3, 2017. [Online]. Available: https://akizukidenshi.com/goodsaffix/DHT11_20180119.pdf . Accessed: Sep. 25, 2026.

[3] e-Gizmo Mechatronix Central, "HC-SR501 passive infrared (PIR) motion sensor datasheet," n.d. [Online]. Available: https://github.com/e-Gizmo/HC-SR501-PIR-motion/blob/master/HC-SR501%20Datasheet.pdf . Accessed: Sep. 27, 2026.

[4] Adafruit Industries, "DHT sensor library," GitHub repository. [Online]. Available: https://github.com/adafruit/DHT-sensor-library . Accessed: Sep. 29, 2026.

12. Repository Link and File Structure

GitHub repository: https://github.com/JosephWalusimbi-eng/lab1

lab1/: firmware/sensor_node/ (ESP32 sketch), simulation/ (Wokwi), python/ (acquisition.ipynb, analysis.ipynb, calibrate.py), data/raw and data/processed, figures/, docs/ (circuit_diagram.png, lab_report.pdf) and README.md.